

Experiment 13: Alpha-Beta Pruning Algorithm

Aim

To implement the Alpha-Beta Pruning algorithm using Python.

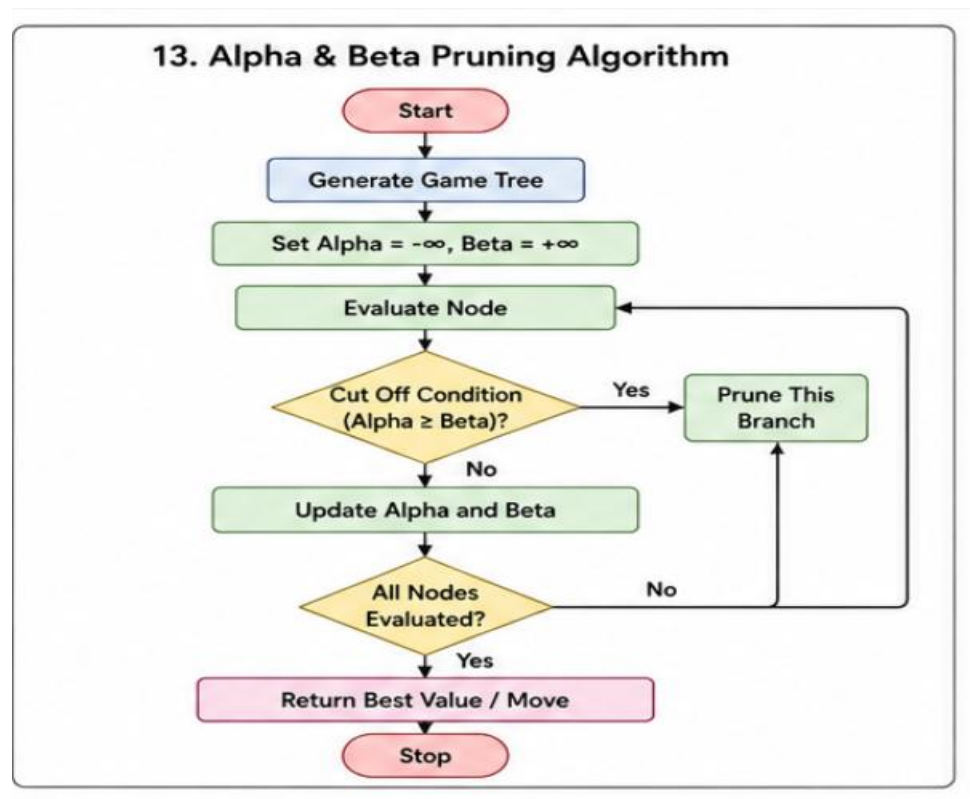
Objective

To improve the efficiency of the Minimax algorithm by eliminating unnecessary branches.

Algorithm

1. Initialize Alpha and Beta values.
 2. Evaluate game states.
 3. Update Alpha and Beta values.
 4. Prune branches when possible.
 5. Display the final result.
-

Flowchart



Code

```
alpha = float('-inf')
```

```
beta = float('inf')
```

```
score = 10
```

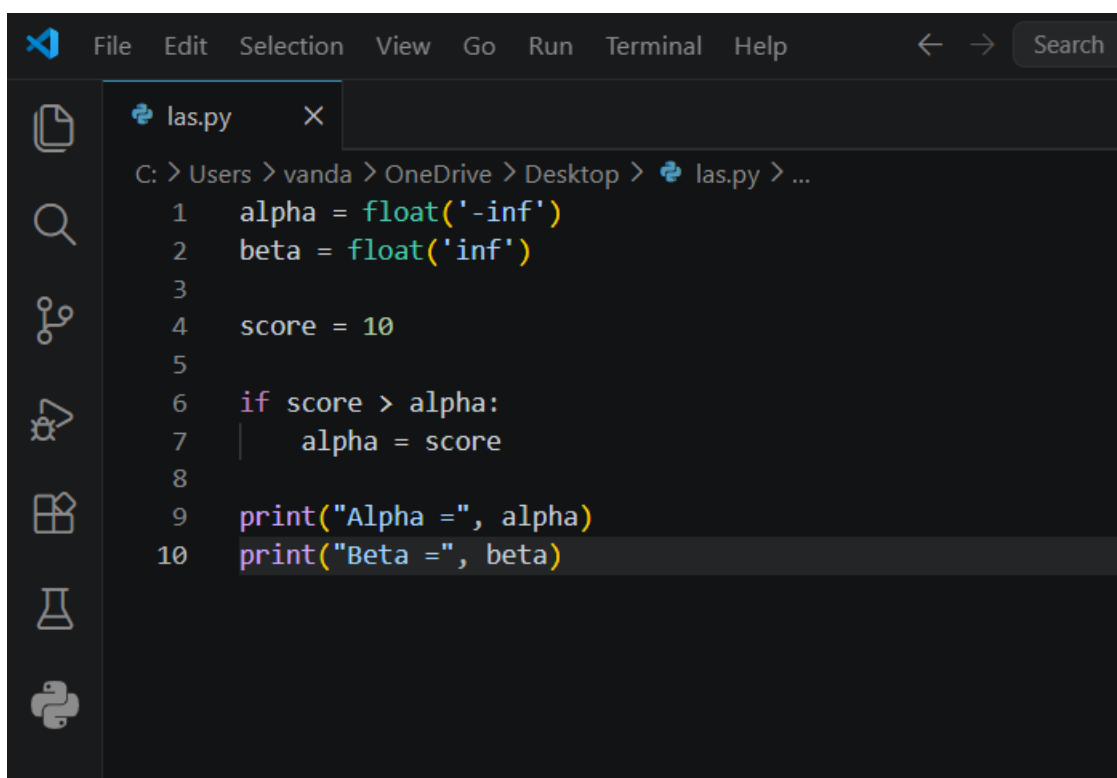
```
if score > alpha:
```

```
    alpha = score
```

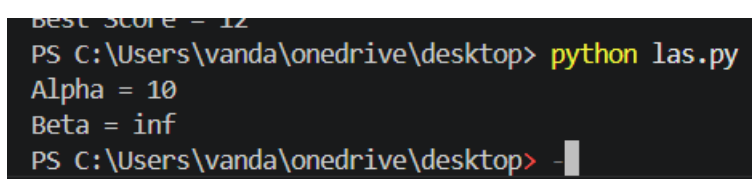
```
print("Alpha =", alpha)
```

```
print("Beta =", beta)
```

Output :

A screenshot of a code editor window with a dark theme. The menu bar at the top includes File, Edit, Selection, View, Go, Run, Terminal, and Help. A search bar is on the right. The left sidebar shows icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, and Python. The main editor area shows a file named 'las.py' with the following code:

```
1  alpha = float('-inf')
2  beta = float('inf')
3
4  score = 10
5
6  if score > alpha:
7      alpha = score
8
9  print("Alpha =", alpha)
10 print("Beta =", beta)
```

A screenshot of a terminal window with a dark background. It shows the command 'python las.py' being executed. The output is:

```
Best Score = 12
PS C:\Users\vanda\onedrive\desktop> python las.py
Alpha = 10
Beta = inf
PS C:\Users\vanda\onedrive\desktop> -
```

Result

The Alpha-Beta Pruning algorithm was executed successfully.

Conclusion

Alpha-Beta Pruning reduces the number of nodes evaluated, making the Minimax algorithm faster.